Day 83 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 7 reference

Watch: Coding a Transformer from scratch, training and inference (Umar Jamil) | Let's build GPT: from scratch, in code, spelled out (Andrej Karpathy)

`day83_tiny_transformer.py`, segment by segment, in the order `main()` uses things. Section 1 is the six exercise scripts' functions, copied verbatim (explained in Days 77–82's guides); this reference covers what is new: decoding, training and the experiments. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: P = every weight, in nested dictionaries (Day 81's exercise) · G = the gradients, flattened by name · GO = 9, EOS = 0 · exact match = every digit and EOS right

Segment 0 — Setup

Settings, and Day 71's numbers

CODE

```
SEEDS = [0, 1, 2]
D, F, HEADS = 32, 64, 4
LR, STEPS = 0.5, 20000
CHECKS = (500, 1000, 1500, 2000, 2500, 5000, 7500, 10000, 15000, 20000)
EVAL_LENGTHS = (4, 8, 12, 16, 20)
DAY71 = dict(weights=17866, n20_2500="0.000", n20_5000="0.992", tok25="0.742",
             checks=[2500, 5000, 7500, 10000, 12500, 15000, 17500, 20000], curve20=[0.0, 0.992, 1,
                 1, 1, 1, 1, 1])
```

- `CHECKS = (500, 1000, ...)` — denser early checks than Day 71's every 2500 steps: the Transformer learns fast.
- `DAY71 = dict(...)` — Day 71's results, copied from its run output, so the comparison is printed beside the new numbers.

Segment 1 — The parts

How the parts are copied in

CODE

```
# ---- from day77_exercise.py, verbatim ----
def sinusoidal_encoding(n, d):
    p = np.arange(n)[:, None]
    i = np.arange(d // 2)[None, :]
    angle = p / 10000 ** (2 * i / d)
    PE = np.zeros((n, d))
    PE[:, 0::2], PE[:, 1::2] = np.sin(angle), np.cos(angle)
    return PE
```

- `# ---- from day77_exercise.py, verbatim ----` — a banner above each group of functions names the exercise script they come from.
- `def sinusoidal_encoding(n, d)` — Day 77's part, exactly as written there; the other five groups follow the same way.

Segment 2 — Decoding and training

generate and evaluate

MATH

$$\textit{next token} = \textit{argmax } p(y \mid GO, y_1, ..., y_t, \textit{source}), \textit{ fed back in}$$

CODE

```
def generate(P, src, n_steps):
    m = len(src); smask = np.ones(src.shape, bool)
    tok = np.full((m, 1), GO)
    for _ in range(n_steps):
        logp, _ = transformer_forward(src, tok, P, HEADS, smask, np.ones(tok.shape, bool))
        tok = np.concatenate([tok, logp[:, -1].argmax(axis=-1)[:, None]], axis=1)
    return tok[:, 1:]
def evaluate(P, n, task):
    _, _, Y, src = fixed_test(n, task)
    pred = generate(P, src, Y.shape[1])
    return np.all(pred == Y, axis=1).mean(), (pred == Y).mean(), (pred == Y).mean(axis=0)
```

- `logp[:, -1].argmax(axis=-1)[:, None]` — the last position's most probable token, as an (m, 1) column to append.
- `np.all(pred == Y, axis=1).mean()` — exact match: a sequence counts only if every token is right.
- `(pred == Y).mean(axis=0)` — token accuracy at each output position (Section 4).

train

MATH

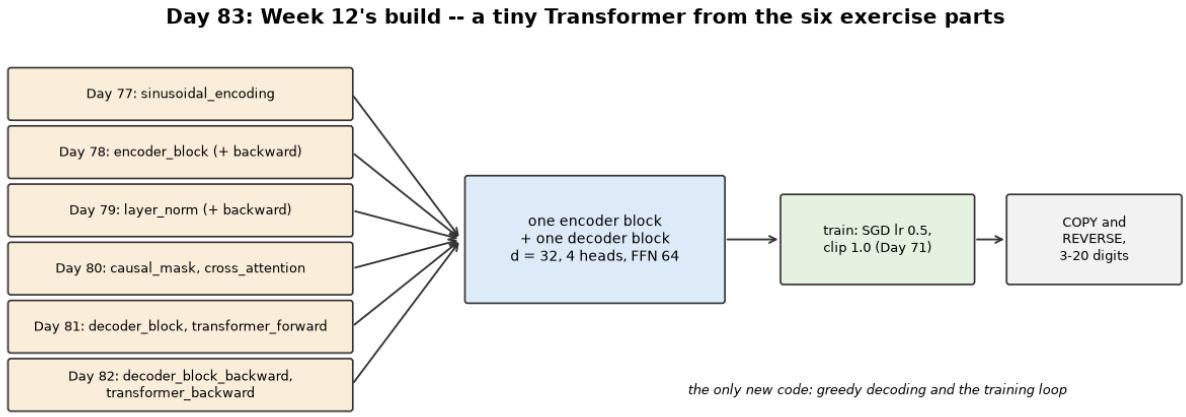
$$w \leftarrow w - lr \cdot \min(1, 1 / |G|) \cdot G \quad \textit{(Day 60's clipping at 1.0, Day 71's rule)}$$

CODE

```
def train(task, seed, lr=LR, steps=STEPS):
    P = init_params(np.random.RandomState(seed), V, D, F)
    rng = np.random.RandomState(seed + 100)
    curve = []
    for step in range(1, steps + 1):
        n = rng.randint(3, 21)
        _, Xd, Y, src = make_batch(rng, 32, n, task)
        tgt_in = Xd.argmax(axis=-1)
        sm, tm = np.ones(src.shape, bool), np.ones(tgt_in.shape, bool)
        logp, cache = transformer_forward(src, tgt_in, P, HEADS, sm, tm)
        G = dict(leaves(transformer_backward(logp, Y, tm, P, HEADS, cache)))
        norm = np.sqrt(sum(np.sum(g ** 2) for g in G.values()))
        for k, A in leaves(P):
            A -= lr * min(1.0, 1.0 / norm) * G[k]
        if step in CHECKS:
            curve.append([evaluate(P, n_, task)[0] for n_ in EVAL_LENGTHS])
    return P, np.array(curve)
```

- `init_params(np.random.RandomState(seed), V, D, F)` — Day 81 exercise's weights, output layer already 10 times smaller.
- `Xd.argmax(axis=-1)` — Day 69's data is one-hot; the exercise model reads ids.
- `dict(leaves(transformer_backward(...)))` — Day 82's nested gradients flattened to 'enc.Wq', 'dec.cross.Wk', ...
- `for k, A in leaves(P): A -= ...` — leaves yields the arrays themselves, so -= updates P in place.

SYSTEM DIAGRAM



Week 12's build: six exercise parts, one trained model.

Segment 3 — Experiments

demo_training: when every length is perfect

CODE

```
for s in SEEDS:
    c = train_cached(task, s)[1]
    done = [CHECKS[i] for i in range(len(CHECKS)) if np.all(c[i] == 1.0)]
    print(f"  seed {s}: {c[-1, -1]:.3f}   all perfect from: {str(done[0]) + ' steps' if done else
        'never'}")
```

- `[CHECKS[i] for i in range(len(CHECKS)) if np.all(c[i] == 1.0)]` — the checks at which every length scored exactly 1.000.
- `str(done[0]) + ' steps' if done else 'never'` — the first such check, or 'never' for an empty list.

demo_longer: inside and beyond the training lengths

CODE

```
res = [evaluate(train_cached(task, s)[0], n, task) for s in SEEDS]
ex_, tok = np.mean([r[0] for r in res]), np.mean([r[1] for r in res])
pos = np.mean([r[2] for r in res], axis=0)
extra = f"   {pos[:20].mean():.3f} / {pos[20:n].mean():.3f}" if n > 20 else ""
```

- `np.mean([r[2] for r in res], axis=0)` — per-position accuracy, averaged over the seeds.
- `pos[:20].mean(), pos[20:n].mean()` — positions 1-20, which training covered, against 21 to n, which it never did.

demo_alignment: reading the weights out of the cache

MATH

entropy = –Σ a log a; target = j (COPY) or 19 – j (REVERSE)

CODE

```
A = cache[6][8][5][:, :20, :]
j = np.arange(20); target = j if task == "copy" else 19 - j
entropy = -(A * np.log(A + 1e-12)).sum(axis=2)
print(f"{task:<9}{(A.argmax(axis=2) == target).mean():>18.3f}{A[:, j,
    target].mean():>18.3f}{entropy.mean():>9.3f}")
```

- `cache[6][8][5]` — transformer_forward's cache, item 6 (the decoder block's), item 8 (cross-attention's), item 5 (its weights).
- `[:, :20, :]` — the 20 digit-writing steps; the EOS step is left out.
- `A[:, j, target]` — for each output digit j, the weight on the source position holding it.

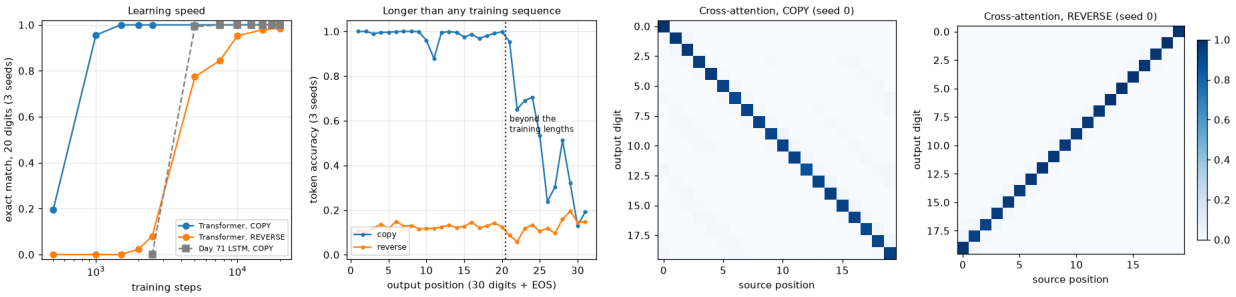
demo_cost: timing Day 71's model in the same run

CODE

```
sys.path.insert(0, os.path.join(HERE, "..", "week-11-attention-mechanism"))
with contextlib.redirect_stdout(io.StringIO()):
    import day71_additive_attention as d71
lstm = d71.AttnSeq2Seq(hidden=32, seed=0)
```

- `sys.path.insert(0, ...)` — lets Python find Day 71's script in Week 11's folder.
- `contextlib.redirect_stdout(io.StringIO())` — anything printed during the import goes into a string, not the output.
- `d71.AttnSeq2Seq(hidden=32, seed=0)` — Day 71's own class: the comparison runs its real code.

REAL GRAPH



Real graph from the script: learning speed, beyond 20 digits, the two cross-attention maps.

Quick Syntax Glossary — new or reinforced constructs

- `for k, A in leaves(P): A -= x` — update arrays inside nested dictionaries in place.
- `np.all(a == b, axis=1)` — True for rows where every entry matches.
- `x if cond else y` — a conditional expression.
- `sys.path.insert(0, folder)` — import a script from another folder.
- `contextlib.redirect_stdout(f)` — send printed output somewhere else for a block.
- `cache[6][8][5]` — index into nested tuples.

End of Day 83 syntax reference. For the experiments and what they showed, see the Day 83 complete guide.